

Systemic Dynamics and Geometric Formation

Computational Visualization of Dynamic Spherical Structures

Prof. Ana Isabel Castillo

28/05/2026

Abstract

This report presents a computational framework for the dynamic visualization of spherical geometric structures generated through discrete surface assembly processes. The proposed model combines geometric parametrization, nonlinear temporal formation laws, rotational transformations and scientific visualization techniques to create an evolving origami-inspired spherical manifold.

The visualization is implemented in Python using NumPy and Matplotlib, integrating dynamic rendering, transparency effects, rotational operators and progressive patch activation mechanisms. The resulting framework bridges applied mathematics, computational geometry and scientific aesthetics, producing a cinematic mathematical visualization environment.

The project also explores how mathematical structures may be interpreted not only analytically but also visually, transforming abstract equations into dynamic evolving systems.

Contents

1	Introduction	2
2	Geometric Parametrization	2
3	Dynamic Surface Assembly	3
4	Rotational Dynamics	3
5	Patch Expansion Dynamics	4
6	Computational Implementation	4
7	Conclusion	5

1. Introduction

Scientific visualization plays an increasingly important role in modern applied mathematics, computational geometry and quantitative modeling. Beyond numerical computation itself, visualization techniques allow mathematical structures to be interpreted dynamically through spatial and temporal evolution.

This work introduces a computational geometric system capable of generating an evolving spherical manifold through discrete origami-inspired surface elements. Instead of constructing a static sphere directly, the system progressively assembles the structure through time-dependent activation of surface patches.

The resulting framework combines:

- geometric parametrization of spherical surfaces;
- nonlinear temporal formation laws;
- rotational dynamical systems;
- transparency and glow rendering techniques;
- cinematic scientific visualization aesthetics.

The project was implemented entirely in Python using NumPy and Matplotlib animation pipelines.

2. Geometric Parametrization

The sphere is parametrized using spherical coordinates:

$$\mathbf{x}(\theta, \phi) = R(\sin \theta \cos \phi, \sin \theta \sin \phi, \cos \theta) \quad (1)$$

where:

- R denotes the sphere radius;
- $\theta \in [0, \pi]$ is the polar angle;
- $\phi \in [0, 2\pi]$ is the azimuthal angle.

The angular domain is discretized into a structured grid composed of finite surface patches:

$$\Theta_i = \{\theta_1, \theta_2, \dots, \theta_N\} \quad (2)$$

$$\Phi_j = \{\phi_1, \phi_2, \dots, \phi_N\} \quad (3)$$

This discretization generates a tessellated spherical structure resembling origami-like geometric tiles.

3. Dynamic Surface Assembly

The evolving surface is modeled as the union of dynamically activated patches:

$$S(t) = \bigcup_{k=1}^{N(t)} P_k \quad (4)$$

where:

- P_k represents an individual surface patch;
- $N(t)$ denotes the number of active patches at time t .

The temporal evolution law is defined by:

$$N(t) = \lfloor N_{\max} q(t) \rfloor \quad (5)$$

where $N_{\max}$ denotes the total number of patches.

The nonlinear activation function is given by:

$$q_\alpha(t) = \min \left\{ 1, \left(\frac{t}{T} \right)^\alpha \right\}, \quad \alpha > 1 \quad (6)$$

In the computational implementation, the parameter choice was:

$$\alpha = 1.7 \quad (7)$$

This nonlinear exponent produces smoother early-stage emergence followed by accelerated structural completion.

4. Rotational Dynamics

To create continuous spatial evolution, rotational transformations are applied to the sphere.

The rotation matrix is defined as:

$$R(t) = R_z(t)R_y(t)R_x(t) \quad (8)$$

where:

$$R_x(t) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\alpha_x t) & -\sin(\alpha_x t) \\ 0 & \sin(\alpha_x t) & \cos(\alpha_x t) \end{bmatrix} \quad (9)$$

$$R_y(t) = \begin{bmatrix} \cos(\alpha_y t) & 0 & \sin(\alpha_y t) \\ 0 & 1 & 0 \\ -\sin(\alpha_y t) & 0 & \cos(\alpha_y t) \end{bmatrix} \quad (10)$$

$$R_z(t) = \begin{bmatrix} \cos(\alpha_z t) & -\sin(\alpha_z t) & 0 \\ \sin(\alpha_z t) & \cos(\alpha_z t) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (11)$$

These operators generate continuous rotational motion and produce the cinematic dynamic behavior observed in the final visualization.

5. Patch Expansion Dynamics

Before full activation, each patch undergoes a temporary radial expansion effect:

$$\tilde{P}_k(t) = \lambda_k(t)P_k \quad (12)$$

where the expansion coefficient is defined as:

$$\lambda_k(t) = 1 + \beta(1 - L_k(t)) \quad (13)$$

with:

- $\beta > 0$ controlling visual expansion intensity;
- $L_k(t)$ representing local patch activation.

This mechanism creates the visual sensation of origami-like emergence from outside the spherical manifold.

These design choices seek to merge mathematical rigor with cinematic visual aesthetics.

6. Computational Implementation

The complete framework was implemented in Python.

The primary computational libraries include:

- NumPy;
- Matplotlib;
- Matplotlib Animation;
- PathEffects.

The animation pipeline uses frame-by-frame rendering through:

$$\text{FuncAnimation} \quad (14)$$

combined with:

$$\text{FFMpegWriter} \quad (15)$$

for MP4 export generation.

The resulting system produces high-resolution scientific cinematic animations suitable for computational visualization portfolios and mathematical digital art frameworks.

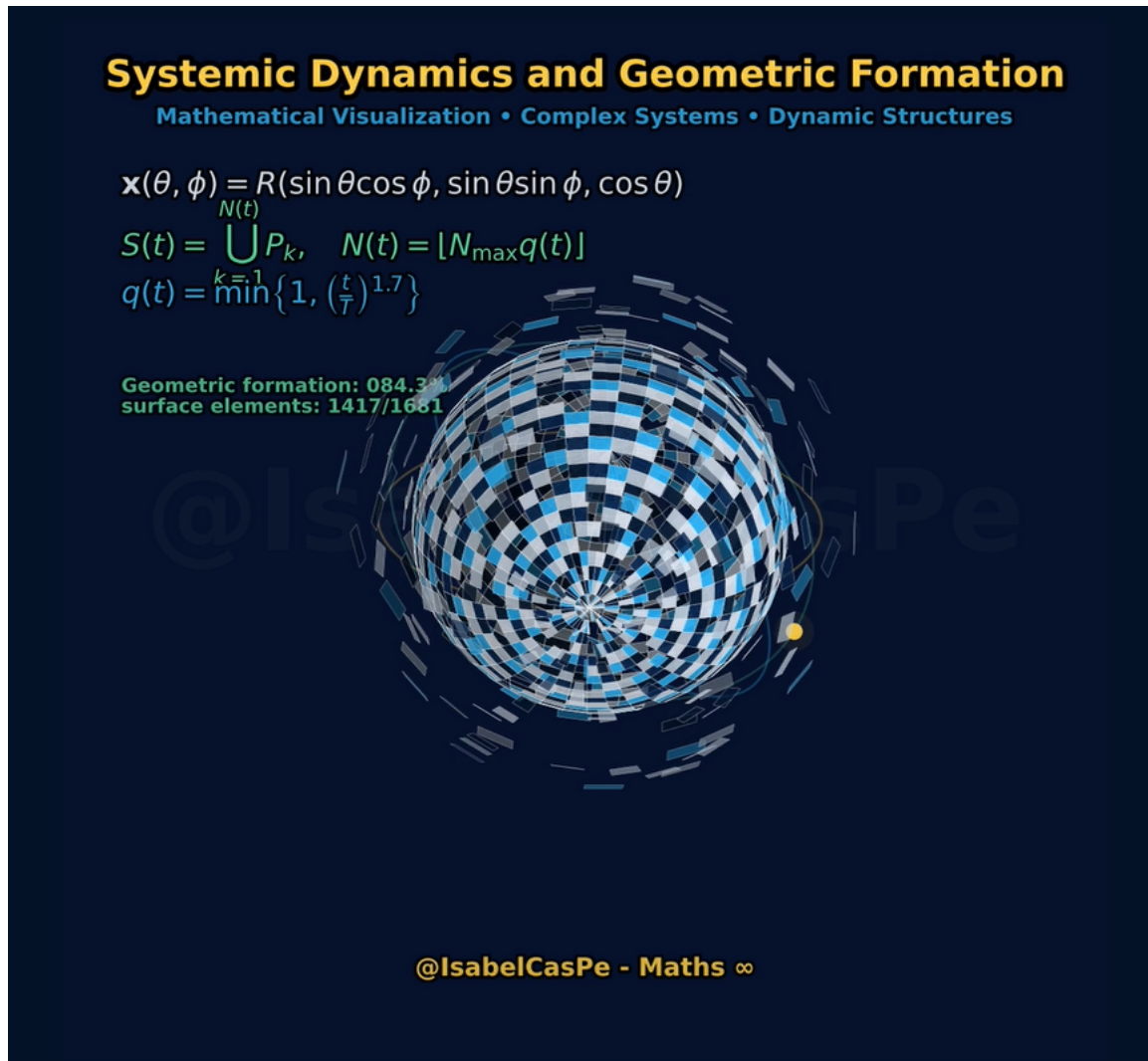


Figure 1: Computational visualization of a dynamically assembled spherical manifold based on discrete surface elements and nonlinear formation laws. The structure evolves through progressive patch activation combined with rotational geometric transformations.

7. Conclusion

This work introduced a mathematical visualization framework for dynamically assembled spherical structures based on discrete geometric surface elements.

The proposed model integrates:

- geometric parametrization;
- nonlinear temporal dynamics;
- rotational transformations;
- computational rendering techniques;
- scientific visualization aesthetics.

The resulting system demonstrates how mathematical structures may evolve beyond static equations into dynamic visual entities capable of communicating geometric and computational behavior through motion.

Future extensions may include:

- machine learning driven surface evolution;
- adaptive network geometries;
- stochastic dynamic perturbations;
- manifold deformation systems;
- financial-network inspired geometric structures.

GitHub Repository

<https://github.com/IsabelCasPe>

@IsabelCasPe – Maths ∞